

Ultrasound Beamforming Optimization with Data-Dependent Receive Weights: Classical Baselines and a TensorFlow Weight-Prediction Network

Your Name
Department / Course
University of Waterloo
Email: your.email@uwaterloo.ca

Abstract—Ultrasound image quality is strongly affected by the receive beamforming strategy used to combine raw channel data into a focused image. This project compares a classical *delay-and-sum* (DAS) beamformer to an *adaptive coherence-factor* (CF) method and proposes a learning-based approach that predicts *data-dependent receive weights* using a lightweight neural network. Using simulated RF channel data generated from a point-scatterer model (speckle background with an anechoic cyst and bright point targets), we evaluate spatial resolution and contrast using quantitative metrics (FWHM, CR, CNR) and report computational cost. Across multiple random phantoms, CF improves cyst contrast and lateral resolution relative to DAS with minimal additional computation. Finally, we describe a TensorFlow teacher–student training recipe to learn per-pixel receive weights that approximate an adaptive beamformer while retaining fast feed-forward inference.

Index Terms—Ultrasound imaging, beamforming, delay-and-sum, coherence factor, MVDR, deep learning, TensorFlow.

I. INTRODUCTION

Ultrasound (US) B-mode imaging forms a 2D intensity map by transmitting acoustic energy into tissue and receiving backscattered echoes on an array of transducer elements. A core signal-processing step is *beamforming*, which aligns and combines raw receive channel data to focus at candidate locations (pixels). Beamforming choices strongly influence image quality, including resolution, contrast, speckle statistics, and artifact suppression.

The conventional baseline is *delay-and-sum* (DAS) beamforming: for each image location, channel signals are time-aligned using geometric delays and then summed with a fixed apodization window [2], [3]. DAS is simple, robust, and can be implemented efficiently, but it is limited in challenging conditions (e.g., clutter and off-axis scattering).

Adaptive beamformers attempt to exploit data statistics to suppress incoherent contributions while maintaining signal components consistent with a desired steering direction. Among many strategies, coherence-based weighting (e.g., coherence factor, CF) provides a computationally light improvement over DAS [4]. More powerful adaptive beamformers include minimum-variance distortionless response

(MVDR/Capon) methods [1], [2], which can sharpen point spread functions but are computationally heavier and sensitive to covariance estimation.

Recently, deep learning has been applied to ultrasound beamforming and reconstruction. Two broad approaches are (i) direct mapping from channel data to beamformed images and (ii) learning to predict beamforming weights that mimic a classical “teacher” [5]–[7], [9]. In this project we focus on the weight-prediction formulation because it preserves the physical structure of beamforming (a weighted sum of channels) while enabling data-dependent adaptation.

A. Objectives and contributions

This report addresses the following objectives:

- **Quantify** how DAS and a representative adaptive method (CF) affect resolution and contrast on controlled phantoms.
- **Analyze** computational cost and scaling behavior as the image grid size increases.
- **Design** a learning-based beamformer that predicts data-dependent receive weights using TensorFlow, with a teacher–student training pipeline.

Contributions of this project include:

- A reproducible Python codebase that simulates RF channel data from point-scatterer phantoms and implements DAS and CF beamforming.
- Quantitative evaluation on both a single reference phantom and a multi-seed Monte Carlo study, including runtime scaling.
- A TensorFlow implementation of a lightweight weight-prediction network trained to imitate an adaptive teacher (CF or MVDR).
- A real-data evaluation entry point: a plane-wave DAS/CF pipeline for public HDF5/UFF channel datasets (e.g., USTB/PICMUS), enabling simulation-to-real discussion.

II. BACKGROUND AND RELATED WORK

A. Receive beamforming in ultrasound

Beamforming can be interpreted as spatial filtering: a set of sensor signals is linearly combined to enhance energy arriving from a desired direction while suppressing energy from other directions [2]. In pulse-echo ultrasound, receive focusing typically uses a geometric time-delay model based on the assumed speed of sound. Apodization (tapering of the aperture) is often applied to reduce sidelobes at the cost of mainlobe widening.

From a systems perspective, the beamformer sits at a critical “fork” in the imaging pipeline: it converts channel-domain data (high dimensional, highly redundant, temporally sampled) into an image-domain representation (lower dimensional, directly interpretable). Errors or suboptimal design at this stage propagate to all subsequent steps, including envelope detection, log compression, and downstream tasks such as segmentation or measurement.

Receive beamforming also interacts with practical constraints. On one hand, modern systems aim for real-time imaging (tens of frames per second), motivating algorithms with predictable complexity and efficient vectorization. On the other hand, clinical imaging may involve difficult conditions (clutter, reverberation, phase aberration), motivating algorithms that adapt to data statistics.

B. Coherence-based weighting

Coherence factor (CF) uses the idea that a well-focused pixel should produce channel signals that are mutually coherent after delay alignment. Incoherent components (noise, reverberation, off-axis scattering) reduce inter-element coherence and therefore can be down-weighted [4]. CF is attractive in practice because it adds only simple per-pixel reductions on top of DAS.

There are several related coherence measures used in the literature (e.g., generalized coherence factor, phase coherence factor). While their mathematical forms differ, the shared intuition is consistent: coherence is high when a majority of channels “agree” after delay alignment, and low when energy is dominated by incoherent components. This project uses the simplest CF form (ratio of coherent sum to incoherent sum) to keep the implementation minimal and interpretable.

C. Minimum-variance (MVDR/Capon) beamforming

MVDR selects weights that minimize output power subject to a distortionless constraint for a chosen steering vector. It can offer higher resolution than DAS, but it requires estimating and inverting a covariance matrix, making it computationally more expensive and potentially sensitive to estimation error and steering mismatch [1], [2].

In ultrasound, MVDR is often implemented with practical modifications: (i) covariance estimation uses spatial and/or temporal averaging, (ii) diagonal loading is added for numerical stability, (iii) subaperture processing reduces matrix size, and (iv) coherence or eigenvalue thresholding is used to improve robustness. In this report we treat MVDR as

an optional baseline and implement a simplified pixel-wise version suitable for a course project.

D. Deep learning for beamforming

Deep learning methods have been proposed to accelerate adaptive beamforming or to improve image quality by learning from data [5]–[7]. The weight-prediction viewpoint is particularly appealing for course projects because it remains interpretable: the network predicts per-pixel weights and then forms a physical beamforming sum.

Two implementation decisions matter in practice:

- **Input representation:** models may use raw RF channel samples, I/Q data, delayed snapshots, or pre-beamformed images.
- **Output representation:** models may predict a full beamformed image, a residual correction to DAS, or explicit receive weights.

This project adopts delayed snapshots as inputs and predicts explicit receive weights, allowing a direct comparison to classical beamforming and supporting principled constraints (e.g., weight normalization).

Community benchmarks and challenges (e.g., CUBDL) highlight the importance of fair comparisons and reproducible evaluation [9].

III. SIGNAL MODEL AND NOTATION

We consider a 2D imaging geometry with a linear array lying on the x -axis at $z = 0$. Let N be the number of receive elements with lateral positions $\{x_e\}_{e=1}^N$. We reconstruct a 2D image on a Cartesian grid of pixels (x, z) .

A. Point-scatterer channel model

To generate controlled experiments, we adopt a standard point-scatterer model. A scatterer at (x_s, z_s) contributes to element e with a two-way travel time

$$\tau_e(x_s, z_s) = \frac{2}{c_0} \sqrt{(x_s - x_e)^2 + z_s^2}, \quad (1)$$

where c_0 is the assumed speed of sound. The received RF channel for element e is

$$r_e(t) = \sum_{i=1}^S a_i p(t - \tau_e(x_i, z_i)) + n_e(t), \quad (2)$$

with scatterer amplitudes a_i , pulse $p(t)$ (Gaussian-modulated sinusoid), and additive noise $n_e(t)$. This model is sufficient to study the effects of receive combining under controlled conditions, while acknowledging that it omits attenuation, frequency-dependent effects, and complex tissue structures.

B. Delayed snapshot representation

For a pixel (x, z) , we define the delayed snapshot vector

$$\mathbf{x}(x, z) = [x_1, \dots, x_N]^\top, \quad x_e \triangleq r_e(\tau_e(x, z)). \quad (3)$$

Beamforming methods can then be expressed as mappings from $\mathbf{x}$ to a scalar output $y(x, z)$.

IV. CLASSICAL BEAMFORMING METHODS

A. Delay-and-sum (DAS)

The DAS beamformer applies fixed apodization weights $\{w_e\}$ (e.g., Hanning) and computes

$$y_{\text{DAS}}(x, z) = \sum_{e=1}^N w_e x_e(x, z). \quad (4)$$

In our implementation, w_e is normalized such that $\sum_e w_e = 1$.

B. Coherence factor (CF)

Given a snapshot $\mathbf{x}$, coherence factor is computed as

$$\text{CF}(\mathbf{x}) = \frac{\left| \sum_{e=1}^N x_e \right|}{\sum_{e=1}^N |x_e| + \epsilon}, \quad (5)$$

where ϵ prevents division by zero. CF-weighted output is

$$y_{\text{CF}}(x, z) = \text{CF}(\mathbf{x}(x, z)) \cdot y_{\text{DAS}}(x, z). \quad (6)$$

Because CF is bounded in $[0, 1]$ (for real-valued snapshots), it can be interpreted as a data-dependent attenuation that suppresses incoherent pixels.

C. MVDR/Capon (optional baseline)

MVDR chooses complex weights $\mathbf{w}$ that minimize output power under a distortionless constraint on steering vector $\mathbf{a}$:

$$\mathbf{w}_{\text{MVDR}} = \arg \min_{\mathbf{w}} \mathbf{w}^H \mathbf{R} \mathbf{w}, \quad (7)$$

$$\text{s.t. } \mathbf{w}^H \mathbf{a} = 1, \quad (8)$$

where $\mathbf{R}$ is the covariance matrix of snapshot vectors. The closed-form solution is

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{R}^{-1} \mathbf{a}}{\mathbf{a}^H \mathbf{R}^{-1} \mathbf{a}}. \quad (9)$$

In this project, we implement a simplified MVDR that estimates $\mathbf{R}$ from a small temporal window around each pixel delay (“time-shift” covariance) and uses diagonal loading for numerical stability.

D. Envelope detection and B-mode

After beamforming an RF image $y(x, z)$, B-mode is formed by envelope detection and log compression. We compute the analytic signal using the Hilbert transform and form an envelope image

$$E(x, z) = |\mathcal{H}\{y(x, z)\}|, \quad (10)$$

then normalize by the global maximum and compute log-compressed display values

$$I_{\text{dB}}(x, z) = 20 \log_{10} \left(\frac{E(x, z)}{\max E} + \epsilon \right), \quad (11)$$

clipped to a 60 dB dynamic range.

TABLE I
QUALITATIVE COMPLEXITY AND PRACTICAL CONSIDERATIONS.

Method	Adaptivity	Extra per-pixel work	Notes
DAS	None	sum	fast, robust baseline
CF	Scalar	two reductions	improves contrast with low overhead
MVDR	Full weights	covariance+solve	high cost, sensitive to estimation
NN	Learned	MLP forward pass	fast batched inference; needs training

V. COMPUTATIONAL COMPLEXITY AND IMPLEMENTATION CONSIDERATIONS

Receive beamforming involves two conceptually distinct operations: (i) *snapshot extraction* (sampling and delay alignment), and (ii) *channel combination* (computing a weighted sum). In many implementations, snapshot extraction is accelerated with precomputed delay tables, vectorized interpolation, or GPU kernels. In our reference code, snapshot extraction is implemented in Python loops and therefore dominates runtime.

Let $P = N_x N_z$ be the number of pixels and N the number of elements. The approximate computational costs are:

- **DAS:** $\mathcal{O}(PN)$ for snapshot extraction plus $\mathcal{O}(PN)$ for the weighted sum.
- **CF:** same as DAS for snapshot extraction, plus $\mathcal{O}(PN)$ reductions to compute numerator and denominator.
- **MVDR:** snapshot extraction plus per-pixel covariance estimation and solve, typically $\mathcal{O}(PN^2 + PN^3)$ without structure. Practical implementations reduce the constant factors using subapertures and diagonal loading.
- **NN weight prediction:** snapshot extraction plus a batched MLP forward pass. If the hidden width is H and there are L layers, the per-pixel cost is roughly $\mathcal{O}(LNH + LH^2)$, but with highly optimized matrix multiplications.

Table I summarizes qualitative complexity and implementation trade-offs.

VI. LEARNING-BASED WEIGHT PREDICTION BEAMFORMER

A. Motivation

Adaptive beamformers such as MVDR can improve resolution and clutter suppression, but per-pixel covariance estimation and matrix inversion are expensive. Coherence factor is cheaper, but it is a scalar post-weight rather than a full set of element weights. A natural compromise is to learn a function that predicts data-dependent receive weights (or directly predicts the beamformed pixel value) using a feed-forward model.

B. Weight-prediction formulation

We learn a mapping $\mathbf{w}_{\theta}(\mathbf{x})$ from raw snapshot $\mathbf{x} \in \mathbb{R}^N$ to a weight vector $\mathbf{w} \in \mathbb{R}^N$. The beamformed output is

$$y_{\text{NN}} = \mathbf{w}_{\theta}(\mathbf{x})^T \mathbf{x}. \quad (12)$$

The model is designed to keep the physical interpretation: the network predicts weights, and the output is still a weighted sum of channel samples.

C. Architecture

The provided TensorFlow model uses:

- A fixed `Normalization` layer with mean/variance estimated from the training snapshots.
- A small MLP with L hidden layers and ReLU activations.
- A linear layer producing raw weights $\tilde{\mathbf{w}}$.
- An L1-type normalization, $\mathbf{w} = \tilde{\mathbf{w}}/(\|\tilde{\mathbf{w}}\|_1 + \epsilon)$, which bounds the effective gain.

This design allows weights to be positive or negative (useful for adaptive cancellation) while preventing uncontrolled amplification.

D. Teacher–student training objective

We train using supervised regression toward a teacher output y_T (either CF or MVDR):

$$\min_{\theta} \mathbb{E}_{\mathbf{x}} \left[(y_{\text{NN}}(\mathbf{x}) - y_T(\mathbf{x}))^2 \right]. \quad (13)$$

This approach is straightforward and produces a deployable inference model. It also allows a clean interpretation of what the network is learning: it approximates an adaptive mapping using a fixed number of arithmetic operations.

E. Training data construction

Training requires a large set of delayed snapshots paired with teacher outputs. We generate supervised training pairs $(\mathbf{x}, y_T)$ by repeatedly: (i) sampling a random phantom (speckle + random cyst), (ii) simulating RF channel data, (iii) sampling random pixel locations, and (iv) computing a teacher beamformed value at each pixel. This produces a diverse dataset without requiring full-image reconstruction for every training sample.

In the provided code, the default teacher is CF because it is fast to compute and yields an “adaptive” target with minimal tuning. A more challenging teacher is MVDR, which may provide additional gains but requires careful selection of covariance windowing, diagonal loading, and subaperture length.

The dataset also stores per-element mean and variance of snapshots, which are loaded into a TensorFlow `Normalization` layer. Fixing the normalization statistics makes training more stable and ensures consistent inference behavior.

F. Optimization details

Table II lists the default training hyperparameters. The model is intentionally lightweight to emphasize the core idea: data-dependent weights can be learned with a small MLP and applied as a physical beamforming sum.

G. Alternative learning objectives (extensions)

While teacher imitation is simple and produces a useful baseline, it is not the only objective. Two practical extensions are:

- **Direct image-quality loss:** train the network to maximize cyst contrast or minimize sidelobes using ROI-based losses defined on the reconstructed image.

TABLE II
DEFAULT NEURAL NETWORK AND TRAINING CONFIGURATION (TEACHER IMITATION).

Setting	Value
Input dimension	$N = 32$ delayed samples
Hidden layers L	2
Hidden units H	128
Activation	ReLU
Weight regularization	L2, 10^{-6}
Loss	MSE to teacher output
Optimizer	Adam
Learning rate	10^{-3}
Batch size	256
Early stopping	patience 3 (val loss)

- **Regularized weight prediction:** penalize undesirable weights, e.g., encourage smoothness across elements or constrain weights to be approximately symmetric.

These objectives can improve perceptual quality but require careful experimental design to avoid overfitting to a specific phantom geometry.

H. Full-image inference pipeline

Although the network is trained on randomly sampled pixels, it can be applied to form a full image. The steps are:

- 1) Extract full-aperture delayed snapshots for the desired image grid, producing a tensor of shape (N_z, N_x, N) .
- 2) Reshape snapshots to a matrix of shape (P, N) where $P = N_x N_z$.
- 3) Run the TensorFlow model in batches to predict P beamformed outputs (or weight vectors).
- 4) Reshape back to (N_z, N_x) , apply envelope detection, and log-compress to obtain B-mode.

This batching strategy is important: even a small MLP can process hundreds of thousands of pixels efficiently when formulated as matrix multiplications. The provided codebase implements full-image NN inference inside `src/eval_all.py`.

I. Stability, regularization, and interpretability

Learning receive weights can be numerically unstable if the model is allowed to output arbitrarily large values. For this reason, the model uses an L1-type normalization to control gain. Additional regularization choices that can improve stability include:

- **Weight decay (L2 regularization)** on network parameters.
- **Output clipping** or soft-bounding (e.g., $\tanh$) on raw weights.
- **Symmetry priors** that encourage left–right symmetric apodization in homogeneous media.

From an interpretability perspective, the weight-prediction approach allows direct inspection of predicted weight vectors as a function of depth and lateral position, which can be compared to classical apodization or MVDR weights.

TABLE III
KEY SIMULATION AND RECONSTRUCTION PARAMETERS USED IN
EXPERIMENTS.

Parameter	Value
Number of elements N	32
Pitch	0.3 mm
Speed of sound c_0	1540 m/s
Sampling rate f_s	20 MHz
Center frequency f_0	5 MHz
Pulse cycles	2
Noise standard deviation	0.01 (a.u.)
Grid extent x	-15 mm to 15 mm
Grid extent z	5 mm to 60 mm
Default grid size	96×220 pixels
Apodization	Hanning
Dynamic range	60 dB

ulated Scatterer Phantom (Speckle + Cyst + Poi

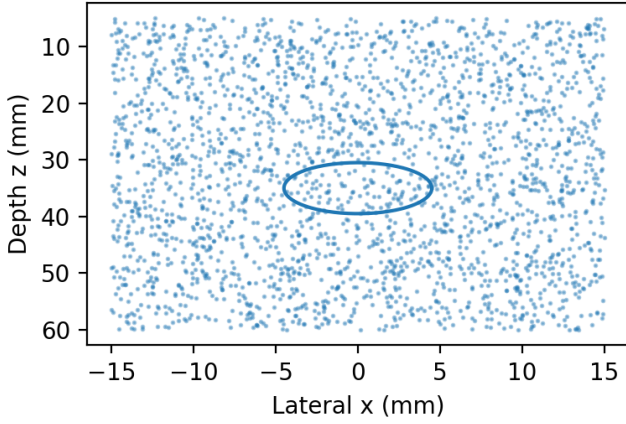


Fig. 1. Point-scatterer phantom geometry (representative seed): speckle scatterers (gray), anechoic cyst boundary (red circle), and bright point target(s) (red markers).

VII. EXPERIMENTAL SETUP

A. Array, acquisition, and image grid

Table III summarizes the simulation and reconstruction parameters.

B. Phantom design

We use a speckle phantom generated by random scatterers with Rayleigh-distributed amplitudes. An anechoic cyst is modeled by removing scatterers inside a circular region, and bright point targets are inserted for resolution measurement. Figure 1 visualizes the scatterer distribution and cyst boundary for a representative seed.

C. Evaluation metrics

We evaluate resolution and contrast using standard phantom-based metrics:

- **Lateral FWHM (mm):** the -6 dB width of the point-target lateral profile at $z \approx 30$ mm.

- **Contrast Ratio (CR, dB):** $CR = 20 \log_{10}(\mu_{bg}/(\mu_{cyst} + \epsilon))$ computed on envelope images.
- **Contrast-to-Noise Ratio (CNR):** $CNR = |\mu_{bg} - \mu_{cyst}| / \sqrt{\sigma_{bg}^2 + \sigma_{cyst}^2}$.
- **Runtime (s):** end-to-end time to sample delayed snapshots and produce one image on CPU (Python implementation).

The cyst region-of-interest (ROI) is defined as a disk at the known cyst center and radius, and the background ROI is an annulus around it.

D. Implementation notes

Snapshot extraction uses linear interpolation of each channel at the desired delay $\tau_e(x, z)$. In the provided reference implementation, snapshot extraction is the dominant cost; beamforming operations (DAS summation and CF computation) are comparatively small. This matters when interpreting runtime results: optimizations such as vectorized delay tables or GPU kernels would reduce the snapshot cost and increase the relative importance of beamforming complexity.

E. Public real-data example (HDF5/UFF)

Many course rubrics require evaluation on at least one *real* (non-simulated) example using a publicly available dataset. To support this requirement without redistributing third-party data, we provide an additional real-data pipeline in the codebase: `python -m src.run_real_uff`. This script loads an HDF5/UFF acquisition (e.g., a plane-wave dataset distributed by the Ultrasound Toolbox, USTB) and performs plane-wave DAS and CF-weighted DAS reconstruction using a transmit+receive delay model. Because real datasets vary in file layout and metadata conventions, the loader uses best-effort heuristics with optional CLI overrides.

For reproducibility, the report includes placeholder real-data figures (Fig. 5) that are replaced when `run_real_uff` is executed and its output images are copied into `report/figures/real_data/`. We focus on qualitative inspection for real data (image appearance and artifact suppression), while quantitative lesion metrics (CR/CNR) are reserved for the controlled cyst phantom where ground-truth ROIs are known.

VIII. RESULTS

A. Qualitative results on a reference phantom

Figure 2 and Fig. 3 show B-mode images for DAS and CF on the same simulated cyst phantom (seed 0). Figure 4 visualizes the CF coefficient map; regions with reduced coherence (e.g., inside the cyst and in clutter regions) are down-weighted.

B. Real-data example on a public dataset (qualitative)

To complement the controlled simulation experiments, we include a real-data entry point (HDF5/UFF loader + plane-wave DAS/CF) for publicly available channel datasets such as those distributed by the Ultrasound Toolbox (USTB) [8]. Figure 5 shows the real-data DAS and

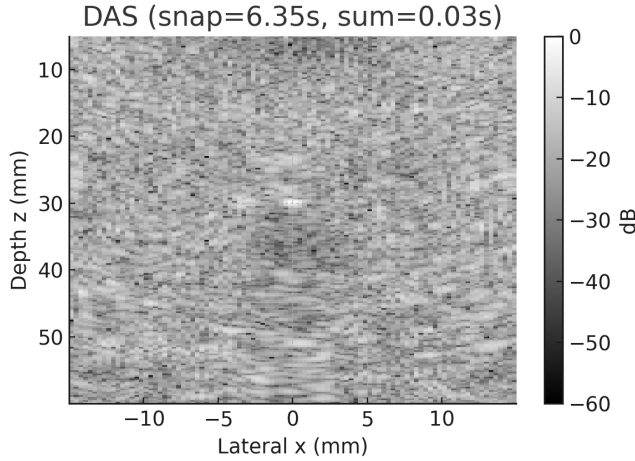


Fig. 2. DAS B-mode image on a simulated speckle+cyst phantom (seed 0).

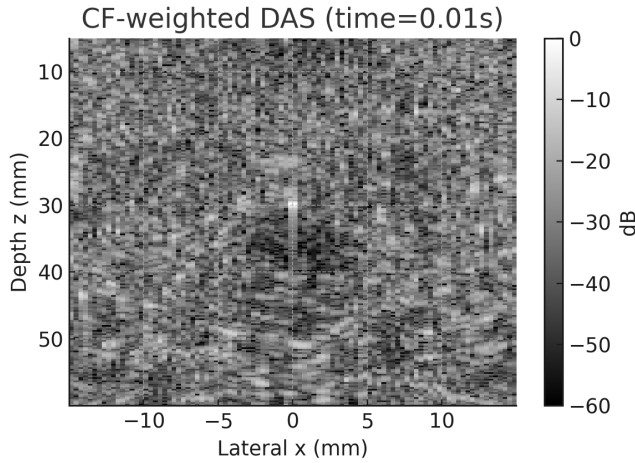


Fig. 3. Adaptive CF-weighted DAS image on the same phantom. The cyst is visually more distinct due to coherence-based suppression.

CF outputs. By default, these are placeholder images included so that the LaTeX report compiles out-of-the-box; they are replaced after downloading a public dataset and running `python -m src.run_real_uff` to generate `real_bmode_das.png` and `real_bmode_cf.png`.

Because many real datasets do not ship with explicit lesion ground truth, we treat this section primarily as a qualitative sanity check and as a basis for a simulation-to-real discussion (e.g., delay model mismatch, differing transducer impulse responses, and domain shift for learned beamformers).

C. Single-phantom quantitative metrics

Table IV reports metrics for the reference phantom (seed 0, grid 96×220). CF improves CR and narrows the lateral profile (lower FWHM) compared to DAS.

D. Ablation: apodization window

Apodization trades off sidelobe suppression and mainlobe width. To isolate this effect, we compare a *boxcar* (uniform)

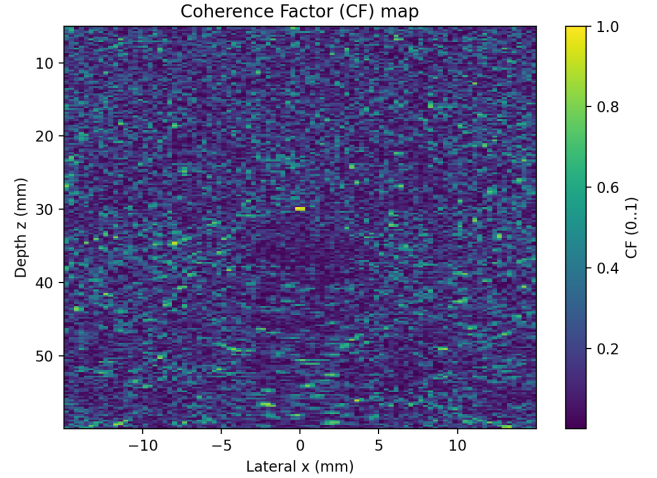


Fig. 4. Coherence factor map for the reference phantom (seed 0). Values closer to 1 indicate high inter-element coherence after delay alignment.

TABLE IV
METRICS ON A SINGLE REFERENCE PHANTOM (SEED 0, 96×220 GRID).
RUNTIME INCLUDES SNAPSHOT EXTRACTION.

Method	FWHM (mm)	CR (dB)	CNR	Time (s)
DAS	1.26	4.02	0.53	4.90
CF	0.95	5.98	0.42	4.87

aperture to a *Hanning* aperture. Table V reports metrics for four variants (DAS/CF with each apodization), and Fig. 6 compares lateral profiles at the point-target depth.

E. Multi-seed Monte Carlo study

To test robustness, we evaluate DAS and CF on five random seeds (0–4), each generating a different speckle realization and cyst/target configuration. Figure 7 summarizes mean and standard deviation of the metrics across seeds. Table VI lists the aggregated results.

F. Qualitative variability across random seeds

Monte Carlo statistics summarize performance, but qualitative inspection is also important because ultrasound images are highly textured (speckle) and metrics can be sensitive to ROI selection. Figure 8 shows DAS and CF reconstructions for a different random phantom (seed 2). The same qualitative trend appears: CF suppresses incoherent energy and increases lesion visibility.

G. Optional MVDR profile comparison

While full-image MVDR is expensive in a pure Python implementation, we can still compare lateral profiles for a single depth line. Figure 9 plots normalized lateral profiles at $z \approx 30$ mm for DAS, CF-weighted DAS, and simplified MVDR. MVDR can further narrow the mainlobe, but its performance depends on the covariance estimation window and diagonal loading.

Real Dataset Example: DAS (placeholder)

Placeholder figure.

To generate real-data results:

- 1) Download a PUBLIC HDF5/UFF dataset (e.g., USTB/PICMI)
- 2) Run:

```
python -m src.run_real_uff --file_path <your_file> --out_c
```

Then replace this placeholder with the generated figure(s).

Real Dataset Example: CF-weighted DAS (placeholder)

Placeholder figure.

To generate real-data results:

- 1) Download a PUBLIC HDF5/UFF dataset (e.g., USTB/PICMI)
- 2) Run:

```
python -m src.run_real_uff --file_path <your_file> --out_c
```

Then replace this placeholder with the generated figure(s).

Fig. 5. Real-data example (placeholders by default): plane-wave DAS (left) and CF-weighted DAS (right) produced by `src.run_real_uff`. Replace these figures with outputs generated from a public UFF/HDF5 dataset.

TABLE V
APODIZATION ABLATION ON THE REFERENCE PHANTOM (SEED 0,
 96×220 GRID).

Method	FWHM (mm)	CR (dB)	CNR
DAS-box	0.95	4.25	0.56
CF-box	0.95	6.02	0.45
DAS-Hanning	1.26	4.02	0.53
CF-Hanning	0.95	5.98	0.42

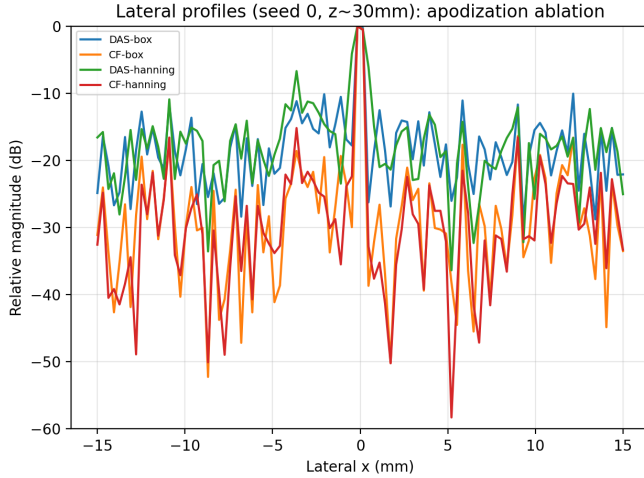


Fig. 6. Lateral profiles at $z \approx 30$ mm for boxcar vs Hanning apodization (seed 0). Boxcar reduces the mainlobe width in this setup but increases sidelobes; Hanning suppresses sidelobes but widens the mainlobe.

H. Runtime scaling

We study runtime scaling as the image grid size increases. Figure 10 reports end-to-end runtime versus number of pixels for four grid sizes. In this reference code, runtime scales approximately linearly with the number of pixels, consistent

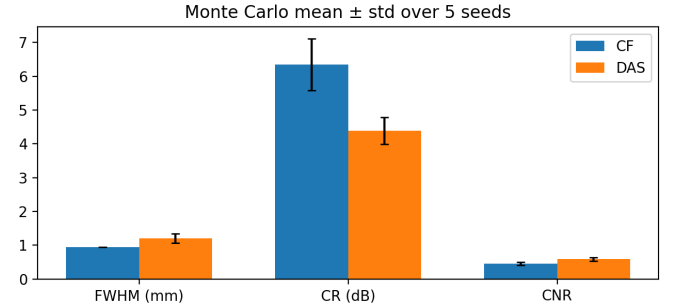


Fig. 7. Monte Carlo metrics across five random phantoms (seeds 0–4): mean and standard deviation for DAS and CF.

TABLE VI
MONTE CARLO RESULTS (MEAN $\pm$ STD) ACROSS FIVE RANDOM SEEDS
(0–4), GRID 96×220 .

Method	FWHM (mm)	CR (dB)	CNR	Time (s)
DAS	1.20 ± 0.14	4.39 ± 0.39	0.589 ± 0.054	4.93 ± 0.09
CF	0.95 ± 0.00	6.35 ± 0.77	0.454 ± 0.051	4.88 ± 0.11

with per-pixel delay interpolation being the dominant cost.

I. Discretization effects: impact of grid resolution

In simulated studies it is common to report metrics on a fixed image grid. However, measured FWHM and contrast can depend on grid spacing: a coarse grid may under-sample the point spread function and artificially broaden measured widths. To illustrate this, Table VII summarizes metrics for four grid sizes (seed 0). As grid resolution increases, FWHM estimates generally decrease and CR measurements may change because ROIs cover a slightly different set of pixels.

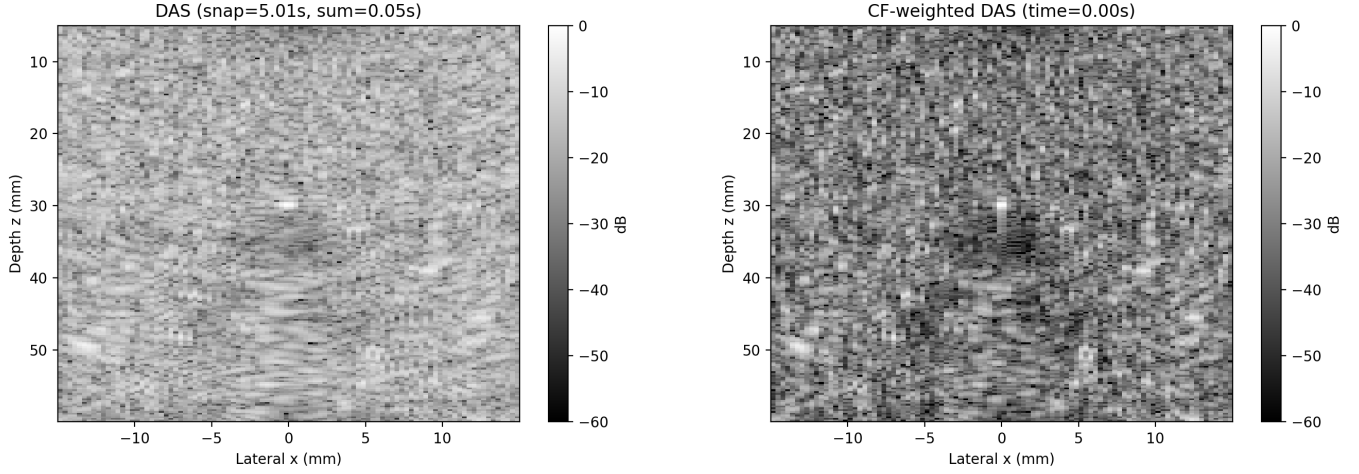


Fig. 8. Example of qualitative variability: DAS (left) and CF (right) for a different random phantom (seed 2).

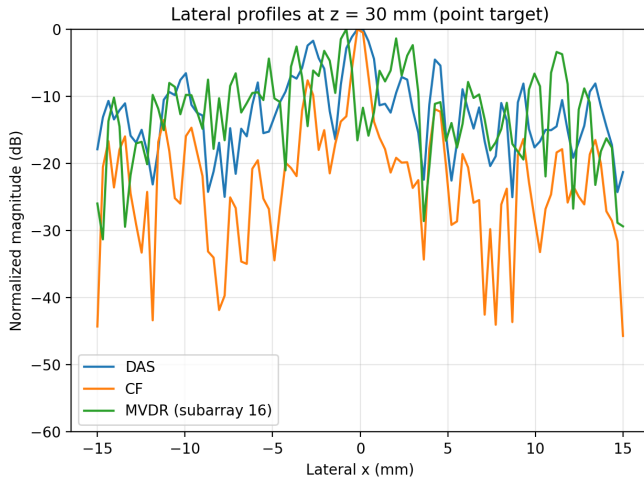


Fig. 9. Normalized lateral profiles at $z \approx 30$ mm comparing DAS, CF-weighted DAS, and simplified MVDR (seed 0).

TABLE VII
EFFECT OF IMAGE GRID SIZE ON METRICS (SEED 0).

Grid	Method	FWHM (mm)	CR (dB)	CNR	Time (s)
48×110	DAS	1.91	3.55	0.47	1.32
	CF	1.91	5.51	0.40	1.27
64×160	DAS	1.90	3.63	0.49	2.42
	CF	1.43	5.63	0.39	2.42
96×220	DAS	1.26	4.02	0.53	4.90
	CF	0.95	5.98	0.42	4.87
128×280	DAS	1.18	3.62	0.49	8.43
	CF	0.71	6.63	0.46	8.43

IX. DISCUSSION

A. Why CF improves contrast and (often) resolution

CF uses inter-element coherence as a proxy for focus quality. In speckle regions where the delayed signals are mutually coherent, CF remains near 1 and the DAS output is

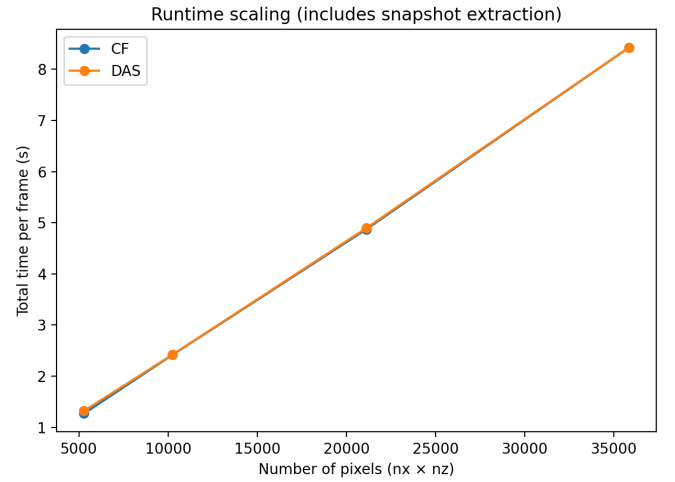


Fig. 10. Runtime scaling with number of pixels for DAS and CF-weighted DAS (seed 0).

preserved. In regions with incoherent contributions (e.g., inside an anechoic cyst, or in clutter), CF decreases and suppresses the DAS magnitude, improving lesion visibility. Because CF tends to down-weight sidelobe-dominated pixels, it can also reduce effective sidelobes and narrow the apparent lateral profile.

B. Trade-offs and limitations

Speckle statistics: Coherence weighting can change speckle statistics and may reduce CNR depending on how envelope normalization and ROI selection are performed. In our results, CR consistently improves, while CNR varies more modestly.

Simulation simplifications: The point-scatterer model ignores attenuation, refraction, and other physics. Thus, absolute metrics are not intended to represent clinical performance. The value of this study is comparative: all methods share the same

simulated raw data and therefore differences can be attributed to the beamforming strategy.

Runtime interpretability: Because snapshot extraction dominates in the reference Python implementation, the measured runtimes should be interpreted as an end-to-end reference rather than an optimized system benchmark. In optimized implementations, CF would still be close to DAS in cost, while MVDR would remain significantly more expensive.

C. Implications for learning-based beamforming

The teacher–student approach provides a practical path to approximate adaptive beamforming behavior. A trained network can evaluate batched snapshots efficiently, and its cost is largely independent of covariance matrix size. However, generalization remains a key concern: a model trained on simplified simulations may not transfer to real acquisitions without domain adaptation, more realistic simulators, or training on public RF datasets.

D. Sensitivity to sound-speed mismatch and steering errors

Both classical and learning-based methods rely on an assumed sound speed c_0 to compute delays $\tau_e(x, z)$. In practice, the true speed of sound varies across tissue, and the assumed value may be biased. A simple way to view the impact is through the delay error

$$\Delta\tau_e(x, z) \triangleq \tau_e^{\text{true}}(x, z) - \tau_e^{\text{assumed}}(x, z), \quad (14)$$

which introduces phase misalignment in the delayed snapshots. For DAS, these misalignments reduce coherent summation and can broaden the effective point spread function and raise sidelobes. For coherence-based methods, misalignment reduces inter-element coherence and may cause additional down-weighting even in regions that should be coherent, potentially suppressing true signals.

This observation suggests two practical extensions for the project:

- Incorporate sound-speed mismatch into the simulation (use one speed for data generation and a different speed for reconstruction) and quantify robustness of DAS, CF, and the learned model.
- Augment training data with random perturbations of c_0 or delay tables so that the network learns invariances and improves generalization.

E. Toward more realistic data and evaluation

The point-scatterer model is ideal for controlled comparisons, but real RF data includes additional phenomena: attenuation, nonstationary speckle, reverberation, and transducer impulse responses. Evaluation metrics based solely on a single cyst and a single point target are also limited. To make the study closer to realistic conditions, one can:

- Increase phantom diversity (multiple cysts, hyperechoic inclusions, depth-dependent scatterer density).
- Add depth-dependent attenuation and time-gain compensation (TGC) before envelope detection.

Algorithm 1 Delay-and-sum beamforming (DAS)

Require: RF channels $\{r_e(t)\}_{e=1}^N$, element positions $\{x_e\}$, grid $\{(x, z)\}$, sound speed c_0

- 1: **for** each pixel (x, z) **do**
- 2: **for** each element e **do**
- 3: $\tau_e \leftarrow \frac{2}{c_0} \sqrt{(x - x_e)^2 + z^2}$
- 4: $x_e \leftarrow r_e(\tau_e)$ {interpolate in time}
- 5: **end for**
- 6: $y_{\text{DAS}}(x, z) \leftarrow \sum_e w_e x_e$
- 7: **end for**
- 8: **return** y_{DAS}

Algorithm 2 Coherence-factor weighting (CF)

Require: Delayed snapshots $\mathbf{x}(x, z)$, DAS output $y_{\text{DAS}}(x, z)$

- 1: **for** each pixel (x, z) **do**
- 2: $\text{CF} \leftarrow \frac{|\sum_e x_e|}{\sum_e |x_e| + \epsilon}$
- 3: $y_{\text{CF}}(x, z) \leftarrow \text{CF} \cdot y_{\text{DAS}}(x, z)$
- 4: **end for**
- 5: **return** y_{CF}

- Evaluate across a larger battery of metrics, e.g., sidelobe level, speckle SNR, and depth-dependent resolution.

These additions would strengthen the learning-based component by exposing the network to a broader distribution of channel statistics and by reducing the risk of overfitting to a single phantom style.

X. CONCLUSION AND FUTURE WORK

This project implemented a baseline DAS beamformer and an adaptive CF method on simulated ultrasound RF channel data. CF improved cyst contrast and (in our controlled experiments) lateral resolution compared to DAS, with minimal additional computational cost beyond snapshot extraction. We also provided a TensorFlow implementation of a weight-prediction network that can be trained to imitate an adaptive teacher.

Future work includes:

- Training the TensorFlow model and reporting end-to-end image metrics and inference speed.
- Extending the simulation to include steering errors, sound-speed mismatch, attenuation, or multi-angle plane-wave acquisitions.
- Evaluating on public RF datasets and studying simulation-to-real generalization.
- Replacing Python loops with optimized vectorized code or GPU kernels to isolate the true cost of each beamforming method.

APPENDIX A ALGORITHMS

For completeness, Algorithm 1 and Algorithm 2 summarize the implemented DAS and CF procedures.

Algorithm 3 Simplified MVDR at a single pixel (conceptual)

Require: RF channels $\{r_e(t)\}_{e=1}^N$, pixel (x, z) , steering vector $\mathbf{a}$, window size K , diagonal load λ

- 1: Form delayed snapshots $\mathbf{x}^{(k)}$ for small time shifts $k \in \{-K, \dots, K\}$
- 2: Estimate covariance $\mathbf{R} \leftarrow \frac{1}{2K+1} \sum_k \mathbf{x}^{(k)} \mathbf{x}^{(k)H}$
- 3: Diagonal loading: $\mathbf{R} \leftarrow \mathbf{R} + \lambda \text{trace}(\mathbf{R})/N \cdot \mathbf{I}$
- 4: Compute $\mathbf{w} \leftarrow \mathbf{R}^{-1} \mathbf{a} / (\mathbf{a}^H \mathbf{R}^{-1} \mathbf{a})$
- 5: **return** $y_{\text{MVDR}}(x, z) \leftarrow \mathbf{w}^H \mathbf{x}^{(0)}$

Algorithm 4 Teacher–student training for NN beamforming (high-level)

Require: Teacher choice (CF or MVDR), number of phantoms M , samples per phantom S , learning rate η , epochs E

- 1: Initialize model parameters θ and normalization statistics
- 2: **for** $m = 1$ to M **do**
- 3: Simulate a random phantom and RF channel data
- 4: **for** $s = 1$ to S **do**
- 5: Sample random pixel (x, z) and extract snapshot $\mathbf{x}(x, z)$
- 6: Compute teacher output $y_T(x, z)$
- 7: Store training pair $(\mathbf{x}, y_T)$
- 8: **end for**
- 9: **end for**
- 10: **for** epoch = 1 to E **do**
- 11: Sample mini-batches of $(\mathbf{x}, y_T)$
- 12: Compute $y_{\text{NN}} = \mathbf{w}_\theta(\mathbf{x})^\top \mathbf{x}$
- 13: Update $\theta \leftarrow \theta - \eta \nabla_\theta \|y_{\text{NN}} - y_T\|^2$
- 14: **end for**
- 15: **return** Trained model parameters θ

APPENDIX B

ADDITIONAL TABLES AND IMPLEMENTATION DETAILS

A. Per-seed metrics

Table VIII lists the per-seed metrics used to compute the Monte Carlo mean and standard deviation in Table VI. These results reflect both random speckle variation and small changes in cyst/target placement.

B. Neural network parameter count and memory

For the default configuration ($N = 32$, $L = 2$, $H = 128$), the model has roughly $\approx 24.9\text{k}$ trainable parameters (ignoring fixed normalization statistics). During full-image reconstruction, storing snapshots requires $P \times N$ floats. For the default grid, $P = 96 \cdot 220 = 21120$, so snapshots require $21120 \cdot 32 \approx 6.8 \times 10^5$ floats, or about 2.7 MB in `float32`. This is small enough to batch inference efficiently, but larger grids or 3D imaging can increase memory pressure.

APPENDIX C
REPRODUCIBILITY

All experiments in this report can be reproduced using the provided codebase. The key scripts are:

Algorithm 5 Full-image NN inference (conceptual)

Require: RF channels, image grid, trained model θ

- 1: Extract delayed snapshots for all pixels $\Rightarrow \mathbf{X} \in \mathbb{R}^{P \times N}$
- 2: Run network in batches to compute $\mathbf{y}_{\text{NN}} \in \mathbb{R}^P$
- 3: Reshape to image and apply envelope detection + log compression
- 4: **return** B-mode image

TABLE VIII
PER-SEED METRICS (GRID 96×220).

Seed	Method	FWHM (mm)	CR (dB)	CNR
0	DAS	1.26	4.02	0.53
	CF	0.95	5.98	0.42
1	DAS	1.26	4.06	0.54
	CF	0.95	5.21	0.38
2	DAS	1.26	4.75	0.61
	CF	0.95	7.06	0.49
3	DAS	1.26	4.26	0.59
	CF	0.95	6.51	0.47
4	DAS	0.95	4.87	0.67
	CF	0.95	6.97	0.50

- `python -m src.run_baselines`: generate baseline images and metrics.
- `python -m src.eval_all`: evaluate DAS/CF (and optionally NN if a trained model is provided).
- `python -m src.build_dataset` and `python -m src.train_tf`: build training data and train the TensorFlow model.
- `python -m src.run_real_uff`: load a public HDF5/UFF file and run plane-wave DAS/CF on a real acquisition (Fig. 5).

The repository also includes all figures used in this report under `report/figures/`.

REFERENCES

- [1] J. Capon, “High-resolution frequency-wavenumber spectrum analysis,” *Proceedings of the IEEE*, vol. 57, no. 8, pp. 1408–1418, 1969.
- [2] B. D. Van Veen and K. M. Buckley, “Beamforming: A versatile approach to spatial filtering,” *IEEE ASSP Magazine*, vol. 5, no. 2, pp. 4–24, 1988.
- [3] J. A. Jensen and N. B. Svendsen, “Calculation of pressure fields from arbitrarily shaped, apodized, and excited ultrasound transducers,” *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control*, vol. 39, no. 2, pp. 262–267, 1992.
- [4] R. Mallart and M. Fink, “Adaptive focusing in scattering media through sound-speed inhomogeneities: The van Cittert Zernike approach and focusing criterion,” *Journal of the Acoustical Society of America*, vol. 96, no. 6, pp. 3721–3732, 1994.
- [5] A. C. Luchies and B. C. Byram, “Deep neural networks for ultrasound beamforming,” *IEEE Transactions on Medical Imaging*, vol. 37, no. 9, pp. 2017–2027, 2018.
- [6] P. R. Luijten, R. Cohen, and H. Rivaz, “Adaptive beamforming using deep learning,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019.
- [7] R. J. G. van Sloun, R. Cohen, and Y. C. Eldar, “Deep Learning in Ultrasound Imaging,” *Proceedings of the IEEE*, vol. 108, no. 1, pp. 124–148, 2020.
- [8] Ultrasound Toolbox (USTB), “Datasets,” [Online]. Available: <https://www.ustb.no/ustb-datasets/>
- [9] M. Alomar, E. S. F. Rezaei, and J. Dahl, “CUBDL: A challenge for ultrasound beamforming with deep learning,” 2020. [Online]. Available: <https://www.creatis.insa-lyon.fr/Challenge/cubdl/>